
Laborationsuppgift 6: Implementering och jämförelse av DFT samt rekursiv och iterativ FFT

Namn 1: Maximilian Andersen

Namn 2: Elias Brännström

Genom att skicka in labbrapporten intygar du/ni att följande regler har följts:

1. Laborationsuppgifter skall lösas självständigt av varje laborationsgrupp. Det är tillåtet att diskutera lösningar, men INTE att kopiera lösningar! Det är alltså INTE tillåtet att ge laborationsresultat eller färdiga lösningar till en annan grupp.
2. Alla gruppmedlemmarna förväntas ta aktiv del i genomförandet av laborationen och skrivandet av rapporten. Detta inkluderar att bygga, programmera, dokumentera, testa och felsöka. Bägge gruppmedlemmarna skall kunna svara på frågor om hur laborationen genomförts och vilka resultat som erhållits.
3. Examination baseras alltid på individuella resultat
4. Sista tillfälle för att redovisa laborationen är i augusti samma år som kursen ges.

Chahed Alasal, juni 2024

Rabaa Khatib, juni 2024

Mats Syde, januari 2024

Tommy Andersson, januari 2020

Introduktion:

I denna laboration kommer du att utforska och jämföra olika metoder för att beräkna diskret Fouriertransform (DFT) och Fast Fourier-transform (FFT). Målet med labben är att:

- Lära dig grunderna i rekursiv och iterativ FFT, samt deras skillnader och användningsområden.
- Använda MATLAB för att implementera rekursiv och iterativ FFT, jämföra dem med MATLAB:s inbyggda fft-funktion och DFT, samt analysera resultaten.
- Förstå prestandafördelar och nackdelar med olika algoritmer för att beräkna Fouriertransform.

Själva laboration är utformat för MATLAB och kodexemplen är specifikt anpassade för MATLAB. Du kan välja att använda ett annat programmeringsspråk, men vi kan inte garantera att vi kan ge hjälp med bibliotek för andra språk.

Syntaxen för MATLAB-funktioner som används för att utföra labbuppgifterna kan hittas i MATLAB-genomgången på Canvas eller genom att använda help-kommandot i MATLAB. Till exempel, om du behöver använda poly-kommandot, skriv 'help poly' i kommandofönstret för en kort beskrivning av hur det används.

Labbrapport

Efter varje avsnitt finns specifika frågor markerade i en röd ruta. Svara på dessa frågor i ett separat dokument. Inkludera all text, de begärda plottarna och koden för varje del i detta dokument, som du sedan lämnar in som din labbrapport.

Förberedelser

Innan du påbörjar labbuppgifterna, se till att besvara de följande förberedande frågorna. Detta kommer att fördjupa din förståelse av ämnet och underlätta arbetet med labbmomenten.

Förberedelsefrågor

- a) Vad är skillnaden mellan iterativ och rekursiv FFT, och hur fungerar de?
- b) Vilka fördelarna finns det med att använda iterativ FFT jämfört med rekursiv FFT?
- c) Vad är syftet med att använda butterfly-algoritmen inom FFT? Förklara stegen som ingår i algoritmen och hur den kombinerar frekvenskomponenter.
- d) Övning för att utföra bit-reversal permutation i FFT:
 - o Skriv ner de ursprungliga indexen för sampelpunkterna: 0, 1, 2, 3, 4, 5, 6, 7.
 - o Skriv om varje index i binärt format och reversera varje binära representation bit för bit.
 - o Konvertera de bitvis-reverserade binära representationerna tillbaka till decimaltal för att få de nya indexen.
 - o Använd de nya indexen för att ordna sampelpunkterna enligt bit-reversal permutation.
 - o Varför är det viktigt att inputsekvensens längd är en tvåpotens för att jämföra FFT-algoritmerna korrekt?
- e) Vad är beräkningskomplexiteten för DFT och FFT?
- f) Hur påverkar antalet element i sekvensen beräkningskomplexiteten för DFT och FFT?

Moment 1 Implementering av rekursiv FFT

I detta moment ska du implementera den rekursiva formen av FFT i MATLAB. Snabb Fourier-transform (FFT) är en algoritm som används för att beräkna den diskreta Fouriertransformen (DFT) med betydligt kortare beräkningstid än vad som krävs vid en direkt beräkning enligt DFT-formeln. Använd följande matris av diskreta indata och följ pseudo-koden nedan för att implementera den rekursiva FFT:

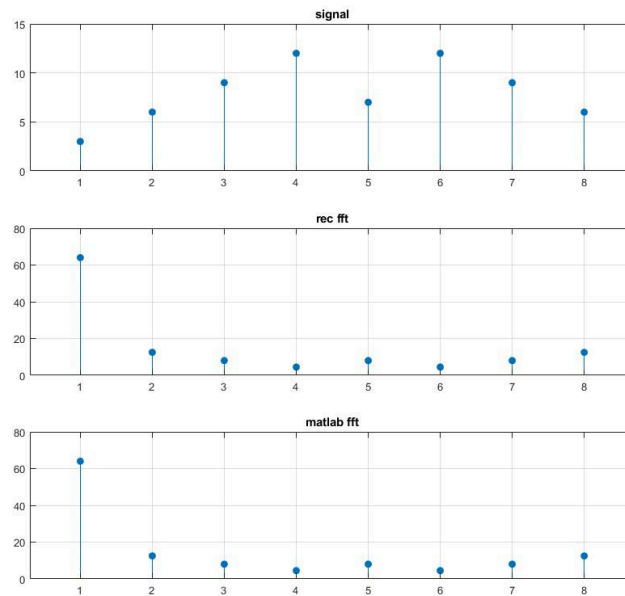
$$x = [3, 6, 9, 12, 7, 12, 9, 6]$$

```
Funktion recursive_fft(inputsekvens):  
    Kontrollera om längden på inputsekvensen är 1, i så fall returnera den.  
  
    Dela upp input_signal i jämna och ojämna element:  
    jämna_del = jämna index från inputsekvensen  
    udda_del = udda index från inputsekvensen  
  
    Rekursiv beräkning av FFT för jämna och udda delar:  
    jämna_fft = recursive_fft(jämna_del)  
    udda_fft = recursive_fft(udda_del)  
  
    Beräkna twiddle-faktorer för nuvarande sekvenslängd:  
    twiddle = exp(-2j * pi * range(0, N/2))  
  
    Kombinera resultaten:  
    För varje index k från 0 till halva längden på inputsekvensen:  
        beräkna: X[k] = jämna_fft[k] + twiddle-faktor[k] * udda_fft[k]  
        beräkna: X[k + halva längden] = jämna_fft[k] - twiddle-faktor[k] *  
        udda_fft[k]  
  
    Returnera X som innehåller Fouriertransformen.
```

1. Skriv en MATLAB-funktion för att implementera den rekursiva FFT-algoritmen med hjälp av den ovanstående pseudokoden*.
2. Beräkna FFT:n av den givna signalen x med den rekursiva FFT-funktion som implementerades i uppgift 1.1.
3. Skriv ut de reella och imaginära delarna av den rekursivt beräknade FFT och inkludera dem i rapporten.
4. Beräkna FFT med MATLAB:s inbyggda funktion $\text{fft}(x)$.
5. Använd sedan $\text{stem}()$ för att plotta följande grafer och infoga dem i rapporten: Den ursprungliga signalen x , FFT-utsignalen för rekursiva FFT, FFT-utsignalen för MATLAB:s inbyggda funktion $\text{fft}(x)$.
6. Analysera och jämför utsignalerna från den rekursiva FFT-funktionen och MATLAB:s inbyggda funktion $\text{fft}(x)$. Dokumentera dina observationer i rapporten.
7. Inkludera MATLAB-koden i rapporten.

* **Viktigt:** Se till att din implementation av den rekursiva FFT-algoritmen hanterar korrekt fall där insignalen har en längd på 1. Om insignalen är av längd 1 ska din funktion enbart returnera den ursprungliga insignalen utan att utföra några ytterligare beräkningar. Om din signal är mer än 1 används if-satsen som en basfall för rekursionen att sluta vid. Var noga med att inkludera detta steg för att undvika felmeddelandet "Out of memory".

1. x
2. x
3. $64.0000 + 0.0000i$ $-12.4853 - 0.0000i$ $-8.0000 + 0.0000i$ $4.4853 + 0.0000i$ $-8.0000 + 0.0000i$ $4.4853 + 0.0000i$ $-8.0000 + 0.0000i$ $-12.4853 - 0.0000i$
4. x



- 5.
6. The output signal from our own recursive fft and the matlab fft is the same.

```
clear all;
x = [3, 6, 9, 12, 7, 12, 9, 6];

x_fft = recursive_fft(x);
disp(x_fft);
x_r_fft = fft(x);
subplot(3,1,1);
stem(x, 'filled');
title('signal');
grid on;
subplot(3, 1, 2);
stem(abs(x_fft), 'filled');
title('rec fft');
grid on;
subplot(3, 1, 3);
stem(abs(x_r_fft), 'filled');
title('matlab fft');
grid on;
```

```
function X = recursive_fft(input_seq)
    N = length(input_seq);

    X = zeros(1, N);

    if(N == 1)
        X = input_seq;
        return;
    end

    even = input_seq(1:2:end);
    odd = input_seq(2:2:end);

    even_fft = recursive_fft(even);
    odd_fft = recursive_fft(odd);

    for k = 1:N/2
        twiddle = exp(-2j * pi * (k-1) / N);
        X(k) = even_fft(k) + twiddle * odd_fft(k);
        X(k+N/2) = even_fft(k) - twiddle*odd_fft(k);
    end
end
```

Moment 2 Implementering av iterativ FFT med butterfly-algoritmen

Den rekursiva FFT, som används i Moment 1 ovan, är minneskrävande och innebär att varje rekursivt anrop lagrar mellanresultat i minnet. Detta kan bli problematiskt för stora indata, eftersom det kräver en betydande mängd minnesutrymme och kan leda till minnesbrist och prestandaproblem. Å andra sidan använder den iterativa versionen av algoritmen en annan strategi som kallas "Butterfly Algorithm". I stället för att använda rekursiva anrop och spara mellanresultat i minnet, beräknar den iterativa algoritmen direkt FFT genom att använda en loop-struktur och uppdatera värdena i en enda iteration. Detta minskar minnesanvändningen och är mer effektivt för stora indata, vilket gör den lämpligare för praktiska tillämpningar där minneskravet är en viktig faktor att beakta. Använd samma matris av diskreta indata som i Moment 1 ovan och följ pseudo-koden nedan för att implementera den iterativa FFT:

Funktion `iterative_fft_with_bit_reversal(inputsekvens)`:

Kontrollera att längden på inputsekvensen är en makt av två, annars avbryt.

Omorganisera inputsekvensen med bit reversal-omordning och lagra resultatet i `X`.

Iterativ FFT-beräkning:

`stegstorlek = 1`

Så länge `stegstorlek` är mindre än längden på inputsekvensen:

Beräkna twiddle-faktorer för det aktuella steget.

För varje startindex i intervallet från 1 till längden på sekvensen med steg om `2 * stegstorlek`:

För varje index `k` från 0 till `stegstorlek - 1`:

Hämta jämna och udda element från sekvensen vid indexen `start + k` och `start + k + stegstorlek`.

Använd twiddle-faktorn för att kombinera jämna och udda element:

beräkna: `X[start + k] = jämnt element + twiddle-faktor * udda element`.

beräkna: `X[start + k + stegstorlek] = jämnt element - twiddle-faktor * udda element`.

Fördubbla `stegstorleken`.

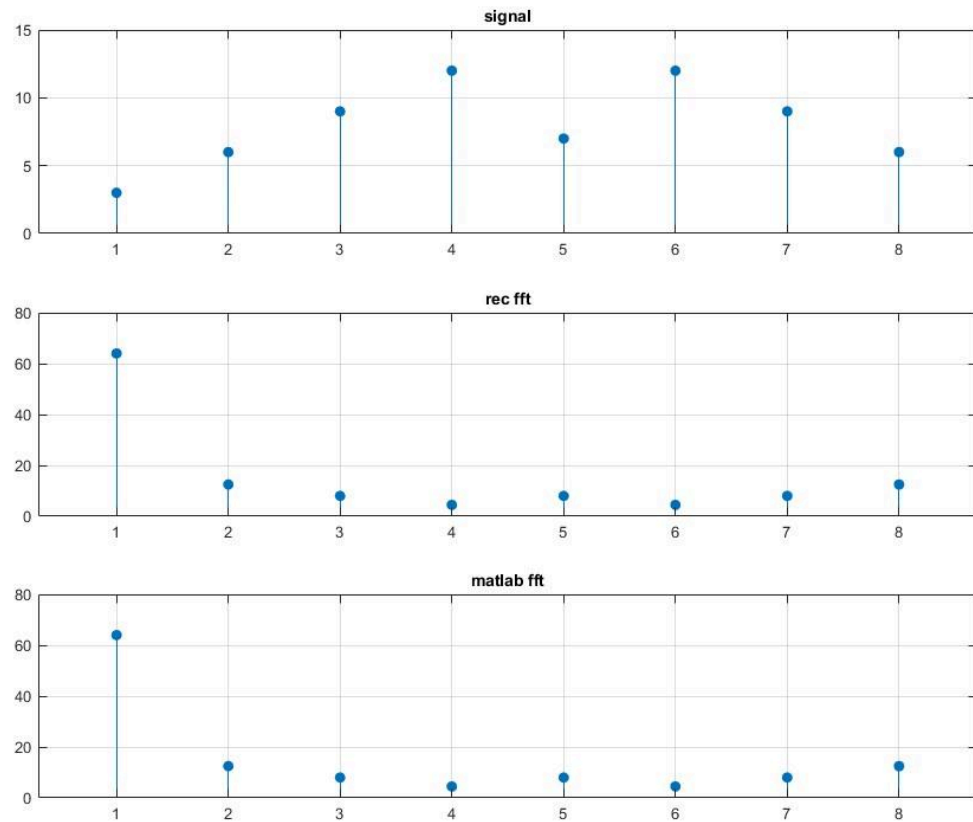
Returnera `X` som innehåller Fouriertransformen.

1. Utför bit reversal-omordning av `x` med hjälp av MATLAB:s inbyggda 'bitrevorder'-funktion.
2. Skriv en funktion enligt den ovanstående pseudokoden för att utföra den iterativa FFT-beräkningen med butterfly-strukturen på den omorganiserade sekvensen från bit reversal.
3. Skriv ut de reella och imaginära delarna av den iterativt beräknade FFT och inkludera dem i rapporten. Jämför resultaten med utdata från uppgift 1.2 i Moment 1.
4. Beräkna FFT med MATLAB:s inbyggda funktion `fft(x)`.
5. Använd sedan `stem()` för att plotta den ursprungliga signalen `x`, utsignalen från iterativ FFT och utsignalen från MATLAB:s `fft`-funktion och infoga dem i rapporten.
6. Analysera och jämför utsignalerna från den iterativa FFT-funktionen och MATLAB:s inbyggda funktion `fft(x)`. Dokumentera dina observationer i rapporten.
7. Inkludera MATLAB-koden i rapporten.

1. `x`
2. `x`
3. `64.0000 + 0.0000i -12.4853 - 0.0000i -8.0000 + 0.0000i 4.4853 + 0.0000i -8.0000 + 0.0000i 4.4853 + 0.0000i -8.0000 + 0.0000i -12.4853 - 0.0000`

The complex and real parts of our iterative and recursive FFT are the identical.

4. `x`



5.

6. The output signal from our own iterative fft and the matlab fft is the same

```
x = [3, 6, 9, 12, 7, 12, 9, 6];

x_fft = iterative_fft_with_bit_reversal(x);
disp(x_fft);
x_r_fft = fft(x);
subplot(3,1,1);
stem(x, 'filled');
title('signal');
grid on;
subplot(3, 1, 2);
stem(abs(x_fft), 'filled');
title('rec fft');
grid on;
subplot(3, 1, 3);
stem(abs(x_r_fft), 'filled');
title('matlab fft');
grid on;
```

```
function X = iterative_fft_with_bit_reversal(input_seq)
    N = length(input_seq);
    if(mod(log2(length(input_seq)), 1) ~= 0)
        return
    end
    X = bitrevorder(input_seq);
    step = 1;
    while(step < N)
        ks = 0:step-1;
        twiddle = exp((-1j * pi * ks) / step);
        for i = 1:2*step:N
            for k = 0:step-1
                even = X(i + k);
                odd = X(i + k + step);
                X(i + k) = even + twiddle(k+1)*odd;
                X(i + k + step) = even - twiddle(k+1)*odd;
            end
        end
        step = step * 2;
    end
end
```

Moment 3 Jämförelse av exekveringstid

Diskret Fouriertransform (DFT) har en beräkningskomplexitet på $O(N^2)$, vilket innebär att antalet beräkningar ökar kvadratisk med antalet element i sekvensen. Detta gör DFT mindre effektiv vid längre sekvenslängder. Både iterativ FFT och rekursiv FFT har en beräkningskomplexitet på $O(N \log \log N)$. Iterativ FFT-algoritmen använder butterfly-strukturen och twiddle-faktorer för att beräkna transformen i steg, vilket gör den betydligt snabbare än DFT. Rekursiv FFT-algoritmen delar sekvensen rekursivt i jämna och udda index och kombinerar deras resultat för att beräkna transformen, vilket ger samma prestanda som iterativ FFT.

I detta moment kommer du att utföra rekursiv FFT, iterativ FFT, DFT, och MATLAB:s inbyggda fft-funktion på inputsekvenser av olika längder och analysera resultatet. Använd de funktioner du skapade i tidigare moment för att beräkna rekursiv FFT och iterativ FFT, samt den funktion du skapade i labb 5 (Moment 2) för att beräkna DFT. Börja med att skapa slumpmässiga inputsekvenser med varierande längder, från 2^4 till 2^{12} . Varje längd bör vara en tvåpotens för att möjliggöra en korrekt jämförelse mellan metoderna.

```
% Definiera längderna på inputsekvensen  
lengths = 2 .^ (4:12); % Från 2^4 till 2^12  
% Antal gånger varje algoritm körs för varje längd  
iterations = 10;
```

För varje längd på inputsekvensen, mät tiden det tar att utföra varje algoritm flera gånger (10 iterationer) för att få genomsnittlig exekveringstid. Använd **tic** och **toc** för att mäta tiden det tar att köra varje algoritm. Följ pseudo-koden nedan för att implementera detta:

Skapa resultatmatriser: `'r_times'`, `'i_times'`, `'d_times'`, `'m_times'` för rekursiv FFT, iterativ FFT, DFT och MATLAB:s `'fft'`.

För varje `'N'` i `'lengths'`:

Skapa inputsekvens `'x'` av längd `'N'`.

Mät tid för rekursiv FFT (`'r_times'`):

Starta `'tic'`.

Kör rekursiv FFT på `'x'` flera gånger.

Sluta `'toc'`.

Lagra genomsnittlig tid i `'r_times'`.

Mät tid för iterativ FFT (`'i_times'`):

Starta `'tic'`.

Kör iterativ FFT på `'x'` flera gånger.

Sluta `'toc'`.

Lagra genomsnittlig tid i `'i_times'`.

Mät tid för DFT (`'d_times'`):

Starta `'tic'`.

Kör DFT på `'x'` flera gånger.

Sluta `'toc'`.

Lagra genomsnittlig tid i `'d_times'`.

Mät tid för MATLAB:s `'fft'` (`'m_times'`):

Starta `'tic'`.

Kör MATLAB:s `'fft'` på `'x'` flera gånger.

Sluta `'toc'`.

Lagra genomsnittlig tid i `'m_times'`.

Skapa resultatmatris:

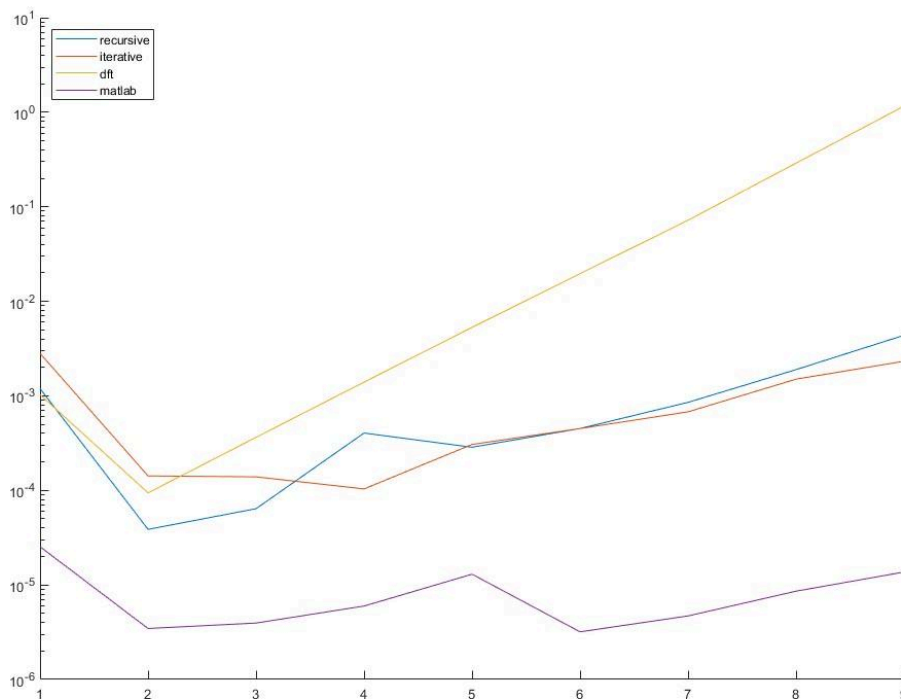
Tabell med `'r_times'`, `'i_times'`, `'d_times'`, `'m_times'`.

|

1. Samla resultaten i en tabell som visar genomsnittlig exekveringstid för varje algoritm och längd på inputsekvensen. Du kan använda MATLAB:s inbyggda funktion `array2table()` för att skapa tabellen.
2. Analysera resultaten och jämför exekveringstiderna för de olika algoritmerna.
3. Diskutera prestanda och effektivitet för varje algoritm
4. Vilken algoritm som är mest respektive minst effektiv för olika längder på inputsekvensen.
5. Inkludera tabellen och MATLAB-koden i rapporten.

1.

Length	Recursive_FFT (s)	Iterative_FFT (s)	DFT (s)	MATLAB_FFT (s)
2^4	0.00052789	0.00721406	0.00023874	2.39E-05
2^5	3.35E-05	4.81E-05	9.72E-05	3.56E-06
2^6	0.00030324	6.38E-05	0.00036191	3.94E-06
2^7	0.00012714	0.00019331	0.00188807	9.16E-06
2^8	0.00044663	0.00041712	0.00515918	1.11E-05
2^9	0.00050687	0.00040881	0.01972245	3.12E-06
2^{10}	0.0007847	0.00061329	0.07256567	4.81E-06
2^{11}	0.00216727	0.00116951	0.28850882	6.57E-06
2^{12}	0.00336315	0.00218145	1.15398961	1.12E-05



2. DFT: The data confirms that the the Direct Fourier Transform has a complexity of $O(N^2)$. We can see that from $N = 1024 \rightarrow N = 2048$ the execution time is quadrupled.

Recursive/Iterative: Both of these algorithms shows expected logarithmic scale and perform significantly better than DFT with longer sequences. At the largest sequences the iterative FFT executes faster than the recursive FFT. This supports the theory that the recursive algorithm suffers from memory overhead and the cost of repeated function calls. Where as the iterative function changes memory in place.

Matlab Builtin: This function performs faster than all of the custom implementations across all input lengths. We don't actually know their implementation but one could think that it uses precompiled, hardware optimized libraries and uses multi threading.

- 3.
4. For short sequences the DFT remains competitive at very short sequences. This probably because of the overhead of bit-reversal and recursive function calls in the custom FFTs outweighs the mathematical efficiency gains.

For longer sequences DFT is the least efficient algorithm. Execution times increase quadratically, making it impractical for large sets.

Overall most efficient: The MATLAB built-in `fft` is the most efficient method for all tested lengths. Among our custom algorithms, the iterative FFT is the most efficient for long sequences due to its lower memory requirements.

```
N = 10;
r_times = zeros(0, N);
i_times = zeros(0, N);
d_times = zeros(0, N);
m_times = zeros(0, N);

lengths = 2 .^(4:12);
iterations = 10;

for i = 1:length(lengths)
    x = randi([1,10],1,lengths(i));

    tic
    for n = 1:iterations
        recursive_fft(x);
    end
    r_times(i) = toc/iterations;

    tic
    for n = 1:iterations
        iterative_fft(x);
    end
    i_times(i) = toc/iterations;

    tic
    for n = 1:iterations
        c_dft(x);
    end
```

```
d_times(i) = toc/iterations;

tic
for n = 1:iterations
    fft(x);
end
m_times(i) = toc/iterations;
end

result_mat = [r_times(:), i_times(:), d_times(:), m_times(:)];
result_t = array2table(result_mat, ...
    'VariableNames',{'Recursive_FFT', 'Iterative_FFT', 'DFT',
    'MATLAB_FFT'});
disp(result_t);
writetable(result_t, "fft_exec_times.csv");

hold on;
yscale log;
plot(r_times);
plot(i_times);
plot(d_times);
plot(m_times);

legend({'recursive', 'iterative','dft', 'matlab'}, 'Location',
'northwest');
hold off;
```